

Rapport de TP - Projet d'Architecture Logicielle AL News

AL News : site d'actualite, services web REST/SOAP et client Python

Etablissement : Ecole Superieure Polytechnique **Formation** : DIT2 Informatique **Enseignant** : M. Diop
Membres du groupe : Mouhamed Sall, Serigne Babacar Sambe, Jacques Boubacar Koukoui **Depot**
GitHub : https://github.com/Amethnb2218/Projet_AL_Groupe_X_Classe **Date** : 9 mai 2026

1. Introduction

Dans le cadre du cours d'architecture logicielle, il nous a ete demande de realiser une application complete autour d'un site d'actualite. Le sujet imposait trois parties principales : un site web de consultation et d'administration, des services web REST et SOAP, puis une application cliente Java ou Python capable d'utiliser ces services.

Notre projet, nomme **AL News**, repond a ce besoin avec une application web developpee en Python avec Flask. Elle permet de consulter des articles, de les classer par categorie, de gerer les contenus selon les profils utilisateurs, d'exposer les donnees au format JSON ou XML, et de gerer les utilisateurs a distance grace a un service SOAP protege par jeton.

2. Objectifs du TP

Le TP avait pour objectif de mettre en pratique les notions d'architecture logicielle vues en cours. Il fallait notamment :

- concevoir un site d'actualite avec page d'accueil, details d'articles, categories et pagination ;
- prendre en compte trois profils : visiteur simple, editeur et administrateur ;
- permettre aux editeurs de gerer les articles et les categories apres authentication ;
- permettre aux administrateurs de gerer les utilisateurs et les jetons d'authentification ;
- creer un service SOAP pour l'authentification et la gestion des utilisateurs ;
- creer un service REST retournant les articles en JSON ou en XML ;
- developper un client Java ou Python utilisant les services web pour administrer les utilisateurs ;
- fournir un code de qualite, structure, maintenable et disponible sur un depot Git public.

3. Technologies utilisees

Le projet utilise les technologies suivantes :

- **Python / Flask** pour le serveur web et les routes applicatives ;
- **SQLite** pour la persistance des donnees ;
- **Jinja2** pour les pages HTML dynamiques ;
- **CSS responsive** pour l'adaptation aux ordinateurs, tablettes et telephones ;

- **XML ElementTree** pour generer les reponses XML et SOAP ;
- **Requests** pour l'application cliente Python ;
- **Pytest** pour les tests automatises.

4. Architecture du projet

Le projet est organise de maniere modulaire afin de separer les responsabilites :

- `app.py` : point d'entree de l'application Flask ;
- `news_app/__init__.py` : creation et configuration de l'application ;
- `news_app/routes.py` : routes web publiques, routes editeur et routes administrateur ;
- `news_app/services.py` : logique metier liee aux articles, categories, utilisateurs et jetons ;
- `news_app/database.py` : connexion SQLite, initialisation et remplissage des donnees ;
- `news_app/api.py` : services REST en JSON et XML ;
- `news_app/soap.py` : service SOAP d'authentification et de gestion des utilisateurs ;
- `client.py` : client Python en console pour administrer les utilisateurs via SOAP ;
- `news_app/templates/` : pages HTML ;
- `news_app/static/` : fichiers CSS, JavaScript et images ;
- `tests/test_app.py` : tests automatises du projet.

Cette organisation montre une separation claire entre la presentation, les services metier, la base de donnees et les interfaces d'integration.

5. Fonctionnalites realisees

5.1 Site web d'actualite

La page d'accueil affiche les articles publies les plus recents avec leur titre, leur description sommaire, leur categorie, leur date et une image. Une pagination avec les boutons **Precedent** et **Suivant** permet de parcourir les articles selon leur anciennete.

Chaque article peut etre consulte en detail en cliquant sur son titre ou sur sa carte. Les articles sont egalement accessibles par categorie. Le site propose une navigation publique permettant aux visiteurs simples de consulter les contenus sans authentification.

5.2 Gestion des profils utilisateurs

Trois profils sont pris en compte :

- **Visiteur simple** : il peut consulter la page d'accueil, les articles, les categories et les services REST publics.
- **Editeur** : il peut se connecter et gerer les articles ainsi que les categories. Il peut lister, ajouter, modifier et supprimer ces donnees.
- **Administrateur** : il dispose des droits de l'editeur et peut aussi gerer les utilisateurs ainsi que les jetons d'authentification SOAP.

Les mots de passe sont haches avec les outils de securite de Werkzeug. Les routes sensibles sont protegees par des controles de role.

5.3 Gestion des articles et categories

Les editeurs et administrateurs peuvent :

- creer, modifier, lister et supprimer des articles ;
- publier ou masquer des articles ;
- associer un article a une categorie ;
- creer, modifier, lister et supprimer des categories ;
- ajouter, remplacer ou retirer des images pour les articles et les categories.

Les slugs sont generes automatiquement afin d'obtenir des URL lisibles et stables.

5.4 Gestion des utilisateurs et des jetons

Les administrateurs peuvent :

- lister les utilisateurs ;
- ajouter un editeur ou un administrateur ;
- modifier les informations d'un utilisateur ;
- supprimer un utilisateur ;
- generer des jetons SOAP ;
- supprimer ou revoquer des jetons.

Des protections evitent la suppression du dernier administrateur et la suppression de son propre compte.

6. Services web REST

Le service REST expose les fonctionnalites de consultation des articles pour des applications externes. Les donnees peuvent etre retournees au format JSON ou XML selon le choix de l'utilisateur avec le parametre `format`.

Les routes principales sont :

- `GET /api/articles` : recuperation de tous les articles publies ;
- `GET /api/articles?format=xml` : recuperation des articles au format XML ;
- `GET /api/articles/grouped` : recuperation des articles regroupees par categories ;
- `GET /api/articles/grouped?format=xml` : meme fonctionnalite au format XML ;
- `GET /api/categories/<id>/articles` : recuperation des articles d'une categorie donnee ;
- `GET /api/categories/<id>/articles?format=xml` : meme fonctionnalite au format XML.

Cette partie respecte la demande du sujet, qui exigeait de fournir les listes d'articles en XML ou JSON.

7. Service web SOAP

Le service SOAP est disponible sur la route `POST /soap`. Il permet :

- `authenticateUser(login, password)` : authentifier un utilisateur avec son login et son mot de passe ;
- `listUsers(token)` : lister les utilisateurs avec un jeton valide ;
- `addUser(token, login, full_name, role, password)` : ajouter un utilisateur ;

- `updateUser(token, id, login, full_name, role, password)` : modifier un utilisateur ;
- `deleteUser(token, id)` : supprimer un utilisateur.

L'accès aux opérations de gestion des utilisateurs exige un jeton généré depuis l'interface d'administration. Cela respecte la contrainte du sujet concernant les services web à accès restreint.

8. Application cliente Python

Le fichier `client.py` contient une application console permettant d'administrer les utilisateurs à partir du service SOAP.

Au lancement, l'application :

- demande l'URL du site ;
- demande le login et le mot de passe ;
- appelle `authenticateUser` pour vérifier l'identité de l'utilisateur ;
- refuse l'accès si l'utilisateur n'est pas administrateur ;
- demande un jeton SOAP administrateur ;
- propose un menu pour lister, ajouter, modifier et supprimer les utilisateurs.

Ce client répond à la partie du TP demandant une application Java ou Python utilisant les services web adéquats.

9. Vérification de conformité au cahier des charges

Exigence du sujet	Réalisation dans le projet	Statut
Depot Git public	Depot GitHub accessible publiquement	Respecte
Groupe de trois étudiants maximum	Groupe composé de trois membres	Respecte
Page d'accueil avec derniers articles	Route <code>/</code> , template <code>home.html</code> , service <code>list_articles</code>	Respecte
Description sommaire des articles	Champ <code>summary</code> affiche sur les listes	Respecte
Boutons suivant et précédent	Pagination sur l'accueil et les catégories	Respecte
Consultation détaillée d'un article	Route <code>/articles/<slug></code>	Respecte
Consultation par catégorie	Routes <code>/categories</code> et <code>/categories/<slug></code>	Respecte
Profil visiteur simple	Accès public aux pages de consultation	Respecte
Profil éditeur	Gestion des articles et catégories après connexion	Respecte
Profil administrateur	Gestion utilisateurs et jetons	Respecte
CRUD articles	Ajout, liste, modification et suppression	Respecte
CRUD catégories	Ajout, liste, modification et suppression	Respecte
CRUD utilisateurs	Ajout, liste, modification et suppression	Respecte
Jetons d'authentification	Génération et révocation dans <code>/admin/tokens</code>	Respecte
SOAP : authentification	Opération <code>authenticateUser</code>	Respecte
SOAP : gestion utilisateurs	Opérations <code>listUsers</code> , <code>addUser</code> , <code>updateUser</code> , <code>deleteUser</code>	Respecte
SOAP protège par jeton	Vérification du jeton avant les opérations protégées	Respecte

Exigence du sujet	Realisation dans le projet	Statut
REST : tous les articles	Route <code>/api/articles</code>	Respecte
REST : articles par categories	Route <code>/api/articles/grouped</code>	Respecte
REST : articles d'une categorie	Route <code>/api/categories/<id>/articles</code>	Respecte
Formats JSON et XML	Parametre <code>?format=json</code> ou <code>?format=xml</code>	Respecte
Client Java ou Python	Client Python <code>client.py</code>	Respecte
Qualite du code	Separation en modules, tests, services dedies	Respecte

Globalement, le projet respecte toutes les grandes exigences fonctionnelles du TP.

10. Tests realises

Une suite de tests automatises a ete ecrite avec Pytest. Elle couvre notamment :

- l'initialisation de la base de donnees ;
- la consultation de l'accueil et des articles ;
- la pagination ;
- les categories ;
- les roles editeur et administrateur ;
- les routes REST en JSON et XML ;
- le service SOAP avec authentication et jeton ;
- la gestion complete des utilisateurs via SOAP ;
- la gestion des images ;
- la protection des pages d'administration.

Commande executee :

```
python -m pytest -q --basetemp .pytest_tmp
```

Resultat obtenu :

```
22 passed in 39.50s
```

Ce resultat confirme que les fonctionnalites principales sont operationnelles.

11. Points forts du projet

Le projet presente plusieurs points forts :

- une architecture claire avec separation entre routes, services, base de donnees, API REST et SOAP ;
- une gestion des roles conforme au sujet ;
- un service REST capable de produire du JSON et du XML ;
- un service SOAP fonctionnel et protege par jeton ;
- un client Python complet pour l'administration des utilisateurs ;
- une interface responsive ;

- une documentation d'installation et de demonstration dans le `README.md` ;
- une suite de tests automatises assez large.

12. Limites et ameliorations possibles

Quelques ameliorations peuvent encore etre envisagees :

- remplacer la cle secrete Flask de demonstration par une variable d'environnement en production ;
- fournir un WSDL plus complet si une validation SOAP stricte est exiguee ;
- ajouter une journalisation des actions d'administration ;
- ajouter une expiration automatique des jetons ;
- enrichir le client Python avec une meilleure gestion des erreurs reseau et une interface plus conviviale.

Ces points ne remettent pas en cause la conformite du TP, mais ils pourraient renforcer le projet dans un contexte de production.

13. Conclusion

Le projet **AL News** realise l'ensemble des parties demandees dans le sujet : site web d'actualite, gestion des profils, administration des contenus, services REST, service SOAP protege par jeton et client Python. L'application est structuree, testee et documentee.

Apres analyse du PDF de consignes, lecture du code et execution des tests, nous pouvons conclure que le travail respecte les exigences du TP d'architecture logicielle.